

Complete Guide to K-Fold & Stratified K-Fold Cross Validation

Here you go  — the **full 100 records, properly split for k=5**.

Notice: every number 1–100 appears **exactly once in a Test set**, and never repeats in the Train set of that same fold.

Fold 1

- **Train (80):** [2, 3, 4, 6, 7, 8, 9, 10, 12, 14, 15, 16, 17, 18, 20, 21, 22, 24, 25, 26, 27, 28, 29, 30, 33, 35, 36, 37, 38, 39, 41, 42, 43, 44, 47, 48, 49, 50, 51, 52, 53, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 72, 73, 75, 76, 79, 80, 82, 83, 85, 86, 87, 88, 89, 90, 92, 93, 94, 95, 96, 97, 98, 99, 100]
 - **Test (20):** [1, 5, 11, 13, 19, 23, 31, 32, 34, 40, 45, 46, 54, 71, 74, 77, 78, 81, 84, 91]
-

Fold 2

- **Train (80):** [1, 2, 3, 4, 5, 7, 8, 9, 11, 13, 14, 15, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 30, 31, 32, 33, 34, 35, 37, 38, 39, 40, 42, 44, 45, 46, 47, 49, 50, 51, 52, 53, 54, 55, 57, 58, 59, 60, 61, 62, 63, 64, 65, 68, 69, 71, 72, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 87, 88, 90, 91, 92, 93, 95, 96, 98, 99, 100]
 - **Test (20):** [6, 10, 12, 16, 17, 27, 29, 36, 41, 43, 48, 56, 66, 67, 70, 73, 86, 89, 94, 97]
-

Fold 3

- **Train (80):** [1, 2, 3, 5, 6, 10, 11, 12, 13, 15, 16, 17, 19, 21, 22, 23, 24, 27, 29, 30, 31, 32, 33, 34, 36, 38, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 64, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 80, 81, 83, 84, 85, 86, 87, 88, 89, 91, 92, 93, 94, 95, 97, 98, 99, 100]
 - **Test (20):** [4, 7, 8, 9, 14, 18, 20, 25, 26, 28, 35, 37, 39, 50, 63, 65, 79, 82, 90, 96]
-

Fold 4

- **Train (80):** [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 37, 38, 39, 40, 41, 43, 45, 46, 48, 50, 52, 53, 54, 56, 61, 63, 64, 65, 66, 67, 70, 71, 72, 73, 74, 75, 77, 78, 79, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 96, 97]
 - **Test (20):** [33, 42, 44, 47, 49, 51, 55, 57, 58, 59, 60, 62, 68, 69, 76, 80, 95, 98, 99, 100]
-

Fold 5

- **Train (80):** [1, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 23, 25, 26, 27, 28, 29, 31, 32, 33, 34, 35, 36, 37, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 54, 55, 56, 57, 58, 59, 60, 62, 63, 65, 66, 67, 68, 69, 70, 71, 73, 74, 76, 77, 78, 79, 80, 81, 82, 84, 86, 89, 90, 91, 94, 95, 96, 97, 98, 99, 100]
 - **Test (20):** [2, 3, 15, 21, 22, 24, 30, 38, 52, 53, 61, 64, 72, 75, 83, 85, 87, 88, 92, 93]
-

✓ This way:

- Train + Test = all 100 records.
 - No overlap between Train and Test within the same fold.
 - Every record is in **exactly one Test set** across the 5 folds.
-

Table of Contents

1. Introduction to Cross Validation
 2. Train-Test Split: The Foundation
 3. K-Fold Cross Validation
 4. Shuffling in K-Fold
 5. Stratified K-Fold Cross Validation
 6. Mathematical Concepts Behind Stratification
 7. Practical Implementation Guide
 8. Common Mistakes to Avoid
 9. Quick Reference Cheat Sheet
-

1. Introduction to Cross Validation

What is Cross Validation?

Cross Validation is a technique used to assess how well a machine learning model will generalize to independent data. It helps us:

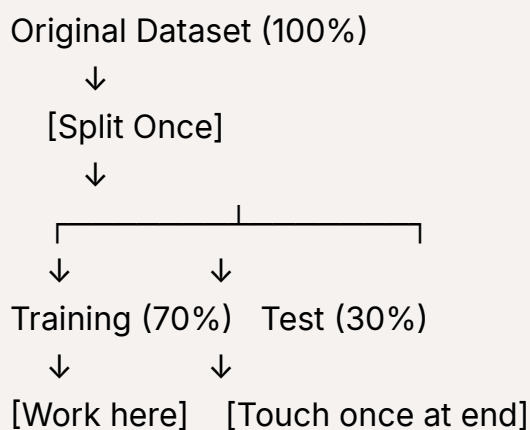
- Evaluate model performance reliably
- Tune hyperparameters
- Compare different models
- Detect overfitting/underfitting

Why Do We Need It?

- A single train-test split might give misleading results
- Different splits can yield different performance metrics
- We want to use our training data efficiently
- We need robust performance estimates

2. Train-Test Split: The Foundation

The Basic Workflow



The Golden Rule

Test data = Sacred vault that remains untouched until final evaluation

Step-by-Step Process

1. Initial Split

- Split entire dataset: 70% training, 30% testing
- Common ratios: 70-30, 80-20, 75-25

2. Set Aside Test Data

- Lock it away
- Do NOT use for any experimentation
- Do NOT use for hyperparameter tuning
- Do NOT use for model selection

3. Work on Training Data

- All cross validation happens here
- All model development happens here
- All hyperparameter tuning happens here

4. Final Evaluation

- Train final model on entire 70% training data
- Evaluate on test data (30%) exactly ONCE
- Report this as final performance

Why This Matters

Analogy: Test set is like your final exam

- You study (train) using practice problems
- You take practice tests (cross validation)
- You take the final exam ONCE
- If you kept retaking the final, you'd memorize answers (data leakage)

3. K-Fold Cross Validation

What is K-Fold?

K-Fold Cross Validation divides your training data into k equal parts (folds) and uses each fold as validation data exactly once.

How It Works

Example: 5-Fold Cross Validation

Training Data (70% of original)



Divide into 5 folds



F1	F2	F3	F4	F5	

Iteration 1: Train on F2,F3,F4,F5 → Validate on F1

Iteration 2: Train on F1,F3,F4,F5 → Validate on F2

Iteration 3: Train on F1,F2,F4,F5 → Validate on F3

Iteration 4: Train on F1,F2,F3,F5 → Validate on F4

Iteration 5: Train on F1,F2,F3,F4 → Validate on F5

Final Score = Average of all 5 validation scores

The Process

1. **Split training data into k folds** (e.g., $k=5$)
2. **For each fold:**
 - Use that fold as validation set
 - Use remaining $(k-1)$ folds as training set
 - Train model and evaluate performance
3. **Average all k performance scores**
4. **This average is your CV score**

Common K Values

- **k=5:** Good balance, commonly used
- **k=10:** More thorough, takes longer
- **k=3:** Faster, less reliable
- **Leave-One-Out (k=n):** Each sample is a fold, very expensive

Advantages

- ✓ Every data point gets used for both training and validation
- ✓ More reliable than single train-validation split
- ✓ Reduces variance in performance estimates
- ✓ Better use of limited data

Formula

CV Score = $(1/k) \times \sum(\text{score}_i)$ for $i=1$ to k

Where:

- k = number of folds
- score_i = performance metric on fold i

4. Shuffling in K-Fold

What is Shuffling?

Shuffling randomly reorders your data before splitting it into folds.

Sequential vs Shuffled

Without Shuffle (Sequential):

Original Data: [A,A,A,A,A,B,B,B,B,B,C,C,C,C,C]

↓

Fold 1: [A,A,A] ← Only class A!

Fold 2: [A,A,B]

Fold 3: [B,B,B] ← Only class B!

Fold 4: [B,B,C]

Fold 5: [C,C,C] ← Only class C!

Problem: Biased folds if data is sorted!

With Shuffle:

Original Data: [A,A,A,A,A,B,B,B,B,B,C,C,C,C,C]

↓ [Shuffle]

↓

Shuffled Data: [B,A,C,A,B,C,A,B,A,C,B,A,C,B,C]



Fold 1: [B,A,C] ← Mixed classes ✓
Fold 2: [A,B,C] ← Mixed classes ✓
Fold 3: [A,B,A] ← Mixed classes ✓
Fold 4: [C,B,A] ← Mixed classes ✓
Fold 5: [C,B,C] ← Mixed classes ✓

Better: More random distribution!

When to Shuffle

✓ ALWAYS SHUFFLE for:

- Classification problems (general)
- Regression problems (non-time series)
- Data that was collected in any particular order
- Data sorted by any feature

✗ DON'T SHUFFLE for:

- Time series data (use TimeSeriesSplit instead)
- Data where temporal order matters
- Sequential prediction problems

Implementation

```
from sklearn.model_selection import KFold

# With shuffle (recommended)
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Without shuffle (only for time series)
kf = KFold(n_splits=5, shuffle=False)
```

Important: random_state

Always set `random_state` when shuffling:

- Makes results reproducible

- Others can verify your work
- You can reproduce your own experiments

```
# Same random_state = Same shuffle every time  
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

5. Stratified K-Fold Cross Validation

What is Stratified K-Fold?

Stratified K-Fold ensures that each fold maintains the same class distribution as the original dataset.

The Problem It Solves

Imbalanced Dataset Example:

- Total: 1000 samples
- Class A: 700 samples (70%)
- Class B: 300 samples (30%)

Regular K-Fold Problem:

Fold 1: 650 A, 350 B (65%-35%) ← Different ratio!
Fold 2: 720 A, 280 B (72%-28%) ← Different ratio!
Fold 3: 680 A, 320 B (68%-32%) ← Different ratio!

Issue: Inconsistent evaluation, unreliable scores

Stratified K-Fold Solution:

Fold 1: 700 A, 300 B (70%-30%) ← Same ratio! ✓
Fold 2: 700 A, 300 B (70%-30%) ← Same ratio! ✓
Fold 3: 700 A, 300 B (70%-30%) ← Same ratio! ✓

Better: Consistent, reliable evaluation!

How It Works: The Algorithm

Step-by-Step Process:

1. Separate data by class

Class A: [600 samples]
Class B: [400 samples]
Total: 1000 samples

2. Calculate proportions

Proportion A = $600/1000 = 0.6$ (60%)
Proportion B = $400/1000 = 0.4$ (40%)

3. Split each class independently

For k=5 folds:
Class A → $600 \div 5 = 120$ per fold
Class B → $400 \div 5 = 80$ per fold

4. Combine to form stratified folds

Fold 1: 120 A + 80 B = 200 (60%-40%) ✓
Fold 2: 120 A + 80 B = 200 (60%-40%) ✓
Fold 3: 120 A + 80 B = 200 (60%-40%) ✓
Fold 4: 120 A + 80 B = 200 (60%-40%) ✓
Fold 5: 120 A + 80 B = 200 (60%-40%) ✓

Visual Comparison

Regular K-Fold (treats all data as one pool):

All Data (1000 samples)
↓
[Random split into 5 parts]
↓
Each fold: 200 random samples
(could be any mix of A and B)

Stratified K-Fold (divides by category):

Class A (600) → [120][120][120][120][120]
 Class B (400) → [80][80][80][80][80]
 ↓ ↓ ↓ ↓ ↓
 Fold 1: 120A+80B Fold 2 Fold 3 Fold 4 Fold 5

Key Differences: Regular vs Stratified

Aspect	Regular K-Fold	Stratified K-Fold
How it splits	Divides whole training data	Divides each class separately
Class distribution	Random (varies per fold)	Fixed (same in all folds)
Best for	Regression, balanced data	Classification, imbalanced data
Shuffling	Optional (set shuffle=True)	Built-in (shuffles by default)
Variance	Higher variance in scores	Lower variance in scores

When to Use Stratified K-Fold

✅ USE STRATIFIED for:

- All classification problems
- Imbalanced datasets (highly important!)
- Binary classification
- Multi-class classification
- When class proportions matter

❌ DON'T USE STRATIFIED for:

- Regression problems (no classes to stratify)
- Time series (use TimeSeriesSplit)

Implementation

```
from sklearn.model_selection import StratifiedKFold

# Stratified K-Fold (shuffles by default)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Usage
```

```
for train_idx, val_idx in skf.split(X, y):  
    X_train, X_val = X[train_idx], X[val_idx]  
    y_train, y_val = y[train_idx], y[val_idx]  
    # Train and evaluate
```

Does Stratification Prevent Overfitting?

No! Common misconception.

Stratification does NOT directly prevent overfitting or underfitting.

What it does:

- Provides more **reliable performance estimates**
- Ensures consistent evaluation across folds
- Prevents evaluation bias from imbalanced folds

Overfitting prevention comes from:

- Regularization
- More training data
- Simpler models
- Dropout, early stopping, etc.

6. Mathematical Concepts Behind Stratification

The Statistical Principle: Stratified Sampling

Stratified sampling is a fundamental statistical technique where the population is divided into homogeneous subgroups (strata), and samples are taken from each stratum proportionally.

Mathematical Foundation

Proportion Preservation:

$$P(\text{class}_i \text{ in fold}_j) = P(\text{class}_i \text{ in training_data})$$

Where:

- P = proportion

- class_i = any class label
- fold_j = any fold

Example:

If training data has 60% class A and 40% class B,
then EVERY fold will have 60% class A and 40% class B

Variance Reduction

Stratification reduces the variance of your cross-validation estimates.

Variance Formula (Concept):

$\text{Var}(\text{Stratified CV}) < \text{Var}(\text{Random CV})$

This means:

- More consistent scores across folds
- More reliable performance estimates
- Better model selection decisions

Why It Works: The Math

Population Representation:

For a dataset with C classes:

Total samples: N
Class c samples: n_c
Proportion: $p_c = n_c / N$

For k folds, each fold j contains:
 $n_{c,j} = n_c / k$ samples from class c

Therefore:
 $p_{c,j} = n_{c,j} / (N/k) = (n_c/k) / (N/k) = n_c/N = p_c$

Conclusion: Proportion maintained! ✓

Expected Value:

$E[\text{Accuracy}_{\text{stratified}}] \approx \text{True_Accuracy}$

With lower variance than random sampling

Sampling Distribution

Regular K-Fold:

- Each fold is a random sample
- High variability between folds
- Performance scores can fluctuate significantly

Stratified K-Fold:

- Each fold is a representative sample
- Low variability between folds
- Performance scores are more stable

Real-World Analogy

Political Polling Example:

Population: 1 million people

- 60% urban
- 40% rural

Bad sampling (like regular K-Fold):

- Survey 1000 random people
- Might get 700 urban, 300 rural (good)
- Or might get 850 urban, 150 rural (biased!)
- Or might get 500 urban, 500 rural (biased!)

Stratified sampling (like Stratified K-Fold):

- Always survey 600 urban, 400 rural
- Maintains 60-40 ratio
- More accurate representation
- More reliable results

7. Practical Implementation Guide

Complete Workflow with Code

```
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 1: Load your data
X, y = load_your_data() # Your features and labels

# Step 2: Initial train-test split (70-30)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,    # 30% for testing
    random_state=42,
    stratify=y        # Stratify the initial split too!
)

# Step 3: Set up Stratified K-Fold on training data
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Step 4: Cross-validation loop
cv_scores = []

for fold, (train_idx, val_idx) in enumerate(skf.split(X_train, y_train)):
    # Split training data into train and validation for this fold
    X_tr, X_val = X_train[train_idx], X_train[val_idx]
    y_tr, y_val = y_train[train_idx], y_train[val_idx]

    # Train model
    model = RandomForestClassifier(random_state=42)
    model.fit(X_tr, y_tr)

    # Validate
    y_pred = model.predict(X_val)
```

```

score = accuracy_score(y_val, y_pred)
cv_scores.append(score)

print(f"Fold {fold+1}: Accuracy = {score:.4f}")

# Step 5: Calculate average CV score
mean_cv_score = np.mean(cv_scores)
std_cv_score = np.std(cv_scores)
print(f"\nMean CV Accuracy: {mean_cv_score:.4f} (+/- {std_cv_score:.4f})")

# Step 6: Train final model on entire training data
final_model = RandomForestClassifier(random_state=42)
final_model.fit(X_train, y_train)

# Step 7: Final evaluation on test set (ONLY ONCE!)
y_test_pred = final_model.predict(X_test)
test_accuracy = accuracy_score(y_test, y_test_pred)
print(f"Final Test Accuracy: {test_accuracy:.4f}")

```

Using GridSearchCV (Automated)

```

from sklearn.model_selection import GridSearchCV

# GridSearchCV automatically uses cross-validation
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15]
}

# Use StratifiedKFold inside GridSearchCV
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='accuracy',
    n_jobs=-1
)

```

```
# Fit on training data only
grid_search.fit(X_train, y_train)

# Best model and parameters
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV score: {grid_search.best_score_:.4f}")

# Evaluate on test set
test_score = grid_search.score(X_test, y_test)
print(f"Test score: {test_score:.4f}")
```

Time Series: Special Case

```
from sklearn.model_selection import TimeSeriesSplit

# For time series data, use TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)

# This maintains temporal order
for train_idx, val_idx in tscv.split(X_train):
    X_tr, X_val = X_train[train_idx], X_train[val_idx]
    y_tr, y_val = y_train[train_idx], y_train[val_idx]
    # Train on past, validate on future
```

8. Common Mistakes to Avoid

Mistake 1: Using Test Data in Cross-Validation

Wrong:

```
# DON'T do this!
kf = KFold(n_splits=5)
for train_idx, val_idx in kf.split(X, y): # Using entire dataset!
    # This includes test data!
```

Right:


```
# Split first, then CV on training data only
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

kf = KFold(n_splits=5)
for train_idx, val_idx in kf.split(X_train, y_train): # Only training!
    # Correct!
```

✗ Mistake 2: Not Shuffling Data

Wrong:

```
# If data is sorted, this gives biased folds
kf = KFold(n_splits=5, shuffle=False)
```

Right:

```
# Always shuffle (except time series)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

✗ Mistake 3: Using Regular K-Fold for Imbalanced Data

Wrong:

```
# Imbalanced data with 90% class A, 10% class B
kf = KFold(n_splits=5) # Some folds might have very few class B!
```

Right:

```
# Use Stratified for classification
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

✗ Mistake 4: Not Setting random_state

Wrong:

```
# Different results every time = not reproducible
kf = KFold(n_splits=5, shuffle=True)
```

Right:

```
# Same results every time = reproducible
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

✗ Mistake 5: Data Leakage

Wrong:

```
# Scaling before split = data leakage!
scaler.fit(X) # Uses information from test set!
X_scaled = scaler.transform(X)
X_train, X_test = train_test_split(X_scaled)
```

Right:

```
# Split first, then scale
X_train, X_test = train_test_split(X)
scaler.fit(X_train) # Only training data!
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

✗ Mistake 6: Touching Test Set Multiple Times

Wrong:

```
# Trying different models on test set
model1.fit(X_train, y_train)
score1 = model1.score(X_test, y_test) # First time

model2.fit(X_train, y_train)
score2 = model2.score(X_test, y_test) # Second time - bad!

# Selecting best model based on test scores = data leakage!
```

Right:

```
# Use CV for model selection
models = [model1, model2, model3]
```

```

best_model = None
best_cv_score = 0

for model in models:
    cv_scores = cross_val_score(model, X_train, y_train, cv=5)
    if cv_scores.mean() > best_cv_score:
        best_cv_score = cv_scores.mean()
        best_model = model

# Now use test set ONCE
best_model.fit(X_train, y_train)
final_score = best_model.score(X_test, y_test) # Only once!

```

✗ Mistake 7: Wrong K Value

Wrong:

```

# k=2 is too small, unreliable estimates
kf = KFold(n_splits=2)

# k=100 with 1000 samples = tiny folds, high variance
kf = KFold(n_splits=100)

```

Right:

```

# k=5 or k=10 are good defaults
kf = KFold(n_splits=5) # Standard choice
kf = KFold(n_splits=10) # More thorough

```

9. Quick Reference Cheat Sheet

Decision Tree: Which CV to Use?

```

Is your data time-series?
├─ YES → TimeSeriesSplit
└─ NO → Is it a classification problem?

```

```
|— YES → StratifiedKFold  
|— NO → KFold (with shuffle=True)
```

Code Templates

Basic Workflow:

```
# 1. Split data  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42, stratify=y  
)  
  
# 2. Choose CV method  
from sklearn.model_selection import StratifiedKFold  
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)  
  
# 3. Perform CV  
from sklearn.model_selection import cross_val_score  
scores = cross_val_score(model, X_train, y_train, cv=cv)  
  
# 4. Train final model  
model.fit(X_train, y_train)  
  
# 5. Evaluate once on test set  
test_score = model.score(X_test, y_test)
```

Key Parameters

Train-Test Split:

```
train_test_split(  
    X, y,  
    test_size=0.3,    # 30% for testing  
    random_state=42,  # Reproducibility  
    stratify=y        # For classification  
)
```

KFold:

```
KFold(
    n_splits=5,      # Number of folds
    shuffle=True,    # Always shuffle!
    random_state=42  # Reproducibility
)
```

StratifiedKFold:

```
StratifiedKFold(
    n_splits=5,      # Number of folds
    shuffle=True,    # Shuffles by default
    random_state=42  # Reproducibility
)
```

Common K Values

K Value	Training % per fold	Validation % per fold	Use Case
3	66.7%	33.3%	Quick experiments
5	80%	20%	Standard choice
10	90%	10%	More thorough
n	100% (all but 1)	1 sample	Leave-One-Out

Performance Metrics Summary

```
from sklearn.metrics import (
    accuracy_score,  # Classification
    precision_score, # Classification
    recall_score,    # Classification
    f1_score,        # Classification
    roc_auc_score,   # Classification (binary)
    mean_squared_error, # Regression
    r2_score         # Regression
)
```

Complete Example: Binary Classification

```

import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Create dataset
X, y = make_classification(
    n_samples=1000,
    n_classes=2,
    weights=[0.7, 0.3], # Imbalanced
    random_state=42
)

# Step 1: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Step 2: Stratified K-Fold CV
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Step 3: Cross-validation
cv_scores = cross_val_score(model, X_train, y_train, cv=skf, scoring='f1')
print(f"CV F1 Scores: {cv_scores}")
print(f"Mean CV F1: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")

# Step 4: Train on full training set
model.fit(X_train, y_train)

# Step 5: Final evaluation on test set
y_pred = model.predict(X_test)
print("\nTest Set Performance:")
print(classification_report(y_test, y_pred))

```

Summary: Key Takeaways

The Big Picture

1. **Always split data first:** 70-30 or 80-20 (training-testing)
2. **Test set is sacred:** Touch it only once at the very end
3. **All CV happens on training data:** Never on test data
4. **Use Stratified K-Fold for classification:** Maintains class balance
5. **Always shuffle:** Except for time series data
6. **Set random_state:** For reproducibility
7. **k=5 or k=10:** Good default choices

The Workflow

1. Load Data
2. Split → Training (70%) + Test (30%)
3. Set aside test data
4. Cross-validation on training data
 - ├ Choose k (5 or 10)
 - ├ Choose method (Stratified for classification)
 - └ Tune hyperparameters
5. Train final model on all training data
6. Evaluate ONCE on test data
7. Report final performance

Remember

✅ DO:

- Use Stratified K-Fold for classification
- Shuffle your data (except time series)
- Set random_state for reproducibility
- Keep test data separate
- Use CV for model selection

❌ DON'T:

- Use test data in cross-validation
 - Touch test data multiple times
 - Forget to shuffle
 - Use regular K-Fold for imbalanced data
 - Skip cross-validation
-

Additional Resources

Scikit-learn Documentation

- Cross-validation: https://scikit-learn.org/stable/modules/cross_validation.html
- Model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

Further Reading

- "An Introduction to Statistical Learning" - Chapter 5: Resampling Methods
- "The Elements of Statistical Learning" - Chapter 7: Model Assessment and Selection

Document created for Data Science students Last updated: September 2025